

AI Server via TCP/IP

Predictive Maintenance and Industry 4.0 Connectivity

1. Problem Scenario Description

Global Motors Corp operates 50 high-precision industrial robots, where every minute of downtime results in significant financial loss.

The objective of this project is to **centralize intelligence**: instead of processing data locally on each robot, data is sent via **TCP/IP network** to a high-performance central server, which runs AI models and returns diagnostics within milliseconds.

Key Challenges

- **Connectivity:**
The server must handle multiple simultaneous connections using **threads**.
- **Accurate Diagnosis:**
The AI model must detect **anomalies before they turn into failures**.
- **Managerial Integrity:**
The alert dashboard must remain consistent, avoiding errors caused by **race conditions**.

2. Machine Learning Model

The system uses an AI model as a **state classifier**, which:

- Receives a feature vector from sensors
- Returns a classification (e.g., “Normal” or “Failure”)

Server Data Flow

1. **Reception:**
The network thread receives data as a string
Example: "45.2, 0.8, 1200"
2. **Preprocessing:**
Converts the string to floats and formats it into a NumPy array
Example: [[45.2, 0.8, 1200]]
3. **Inference:**
The model loaded in memory executes `.predict()`

4. **Action:**

- If result = "**Failure**":
 - Increment global counter (using **Lock**)
 - Send alert back to the robot

3. Development Roadmap

Lesson 1: Networking and Concurrency

Activity 1: TCP Connection (Sockets)

- Create:
 - `sensor_node.py` (client)
 - `central_server.py` (server)



AI Support:

“Generate Python code for a TCP socket that sends a JSON with 3 sensor values.”

Activity 2: Multithreaded Server

- Each client must be handled by an independent **thread**
- Prevents network blocking



AI Support:

“How to create a multithreaded server that identifies each client’s IP?”

Lesson 2: Intelligence and Consistency

Activity 3: Real-Time Inference

- Load `.pkl` model
- Run `.predict()` inside the thread
- Return result to client



AI Support:

“How to load a Random Forest model from `.pkl` and run prediction inside a thread?”

Activity 4: Synchronization with Lock

- Create global variable: `total_alerts`
- Use `threading.Lock()` to ensure consistency



AI Support:

“Explain how Lock prevents lost updates between threads.”

✓ Summary explanation:

A **Lock ensures mutual exclusion**, preventing multiple threads from updating the same variable simultaneously and avoiding data loss.

4. Final Presentation

The team should act as technical consultants and present:

Demonstration

- Connect 3 robots (terminals)
- Show:
 - data transmission
 - real-time classification
 - correctly updated counter

Network Explanation

- Why use **TCP**:
 - Reliability
 - Guaranteed data delivery
- Role of **threads**:
 - Prevent bottlenecks
 - Keep factory operations running continuously

Prompt Engineering

- Explain how **generative AI tools** supported:
 - integration of networking and AI
 - code generation
 - development optimization

